

CMP6202

AI and Machine Learning Project
2023–2024

Car Price Prediction Model

Forename	Surname	Student ID	Instructor Name	ML Model(s) developed
Dilwar	Loshkor	22152964	Khalid Ismail Nouh Elmitwally	Random Forest Linear Regression

Table of Contents

1	Report Introduction	3
1.1	Dataset identification	3
1.2	Supervised learning task identification	4
2	Exploratory Data Analysis	4
2.1	Question(s) identification	4
2.2	Splitting the dataset	4
2.3	Exploratory Data Analysis process and results	5
2.4	EDA conclusions.....	13
3	Experimental Design	14
3.1	Identification of your chosen supervised learning algorithm(s)	14
3.2	Identification of appropriate evaluation techniques	14
3.3	Data cleaning and Pre-processing transformations	14
3.4	Limitations and Options	23
4	Predictive Modelling / Model Development	23
4.1	The predictive modelling process.....	23
4.2	Evaluation results on “seen” data	25
5	Evaluation and further modelling improvements.....	26
6	Conclusion.....	33
6.1	Summary of results.....	33
6.2	Reflection on Individual Learning	35
7	References	36
7.1	Appendix.....	37

1 Report Introduction

In this report I will be creating a model that predicts car prices based on several features. The report will document my processes and experiences for Exploratory Data Analysis, Data-Preprocessing, Model training and evaluation. This model aims to solve a regression task which involves predicting a continuous value. The model can provide insights to buyers, sellers or other applications.

1.1 Dataset identification

My dataset contains features of used cars that can affect pricing such as model year, milage, transmission and more. The dataset was found on <https://www.kaggle.com/datasets/taeefnajib/used-car-price-prediction-dataset>

This data was collected from a website called <https://www.cars.com/> which contains information about real cars that are being sold. It includes real-world data on car features that influence pricing, ensuring reliability and variety.

The dataset contains 11 different columns and has 4009 rows.

Feature Name	Numerical/Categorical	Description
Brand	Categorical	Brand of the car helps identify manufacturer and important for pricing.
Model	Categorical	Specific Model of the car
Model Year	Numerical	Manufacture date of the car
Milage	Numerical	Indicates how much the car has been used
Price	Numerical	Listing price of the car
Transmission	Categorical	Vehicles Specifications related to gear type.
Fuel Type	Categorical	Type of fuel the car used. (Diesel, gasoline etc)
Engine	Categorical	Vehicles Specifications related to engine.
Interior/Exterior Colours	Categorical	Tells you colours of the car
Accident History	Categorical	Clean title shows the legal history of the car
Clean Title	Categorical	Shows if the car has been involved in any incidents.

Figure 1: List of Features

Transmission, Fuel type and engine are categorical columns which describe the vehicles specifications. While they may have an inherent order, I will treat these as nominal variables for simplicity.

Accident history and clean title are both categorical and they can be converted to binary because they only have two possible results

1.2 Supervised learning task identification

I will be using the dataset mentioned earlier to create a model that predicts car prices. This is a regression task as I am predicting a continuous value based off other numerical and categorical features. This is a supervised learning problem where the model learns from labelled data to make predictions on unseen data.

The target variable in this problem is the price column. The real prices in the dataset will be used to train the model.

2 Exploratory Data Analysis

2.1 Question(s) identification

Explored Questions
- How does milage impact car price?
-
- How does model year impact prices?
- Does accident history impact prices?
- Does clean title impact prices? And does it provide useful data for the model to use?
- Does fuel type impact prices?
- Are there any missing values?
- Which columns will I drop?
- What model and evaluation metrics can I use?

Figure 2: Question table

2.2 Splitting the dataset

Splitting the dataset is an essential step in machine learning because it prevents data leakage and ensures a fair evaluation of the model.

I will be splitting the data into two sets. 80% of the data will be Training (seen data) and 20% will be testing (unseen data). Information from the test set will not impact the models learning in any way to ensure a fair evaluation. Having slightly more training data allows the model to learn more before moving onto unseen data. This split will be implemented after the pre-processing is completed.

2.3 Exploratory Data Analysis process and results

First, we will start with simply viewing the dataset using `df.head()`. This will let us know that it has been imported correctly and we can begin.

	brand	model	model_year	milage	fuel_type	engine	transmission	ext_col	int_col	accident	clean_title	price
0	Ford	Utility Police Interceptor Base	2013	51,000 mi.	E85 Flex Fuel	300.0HP 3.7L V6 Cylinder Engine Flex Fuel Capa...	6-Speed A/T	Black	Black	At least 1 accident or damage reported	Yes	\$10,300
1	Hyundai	Palisade SEL	2021	34,742 mi.	Gasoline	3.8L V6 24V GDI DOHC	8-Speed Automatic	Moonlight Cloud	Gray	At least 1 accident or damage reported	Yes	\$38,005
2	Lexus	RX 350 RX 350	2022	22,372 mi.	Gasoline	3.5 Liter DOHC	Automatic	Blue	Black	None reported	NaN	\$54,598
3	INFINITI	Q50 Hybrid Sport	2015	88,900 mi.	Hybrid	354.0HP 3.5L V6 Cylinder Engine Gas/Electric H...	7-Speed A/T	Black	Black	None reported	Yes	\$15,500
4	Audi	Q3 45 S line Premium Plus	2021	9,835 mi.	Gasoline	2.0L I4 16V GDI DOHC Turbo	8-Speed Automatic	Glacier White Metallic	Black	None reported	NaN	\$34,999

Figure 3: `Df.head()` check

```
[5] df['accident'].value_counts()
```



accident		count
None reported		2910
At least 1 accident or damage reported		986

dtype: int64

Figure 4: Accidents Value count

Next, I did a value count check on the accident column. There are only two possible outcomes meaning we can use binary to encode.

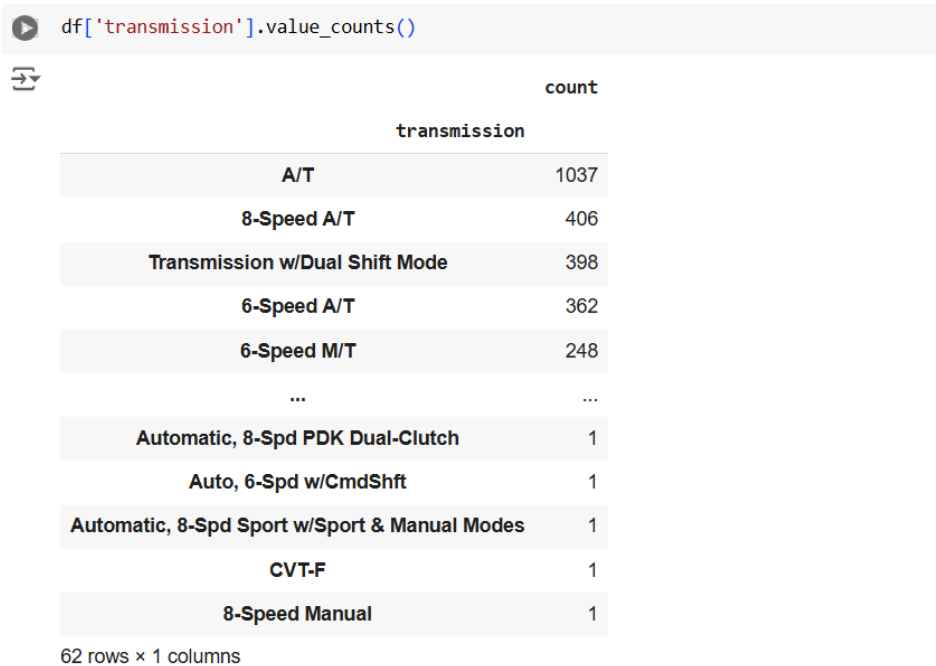


Figure 4: Value counts for transmission column

Similarly, I did a value counts check on the transmission column which has 62 unique values. High cardinality can complicate encoding, so I plan to address this issue by simplifying the data.

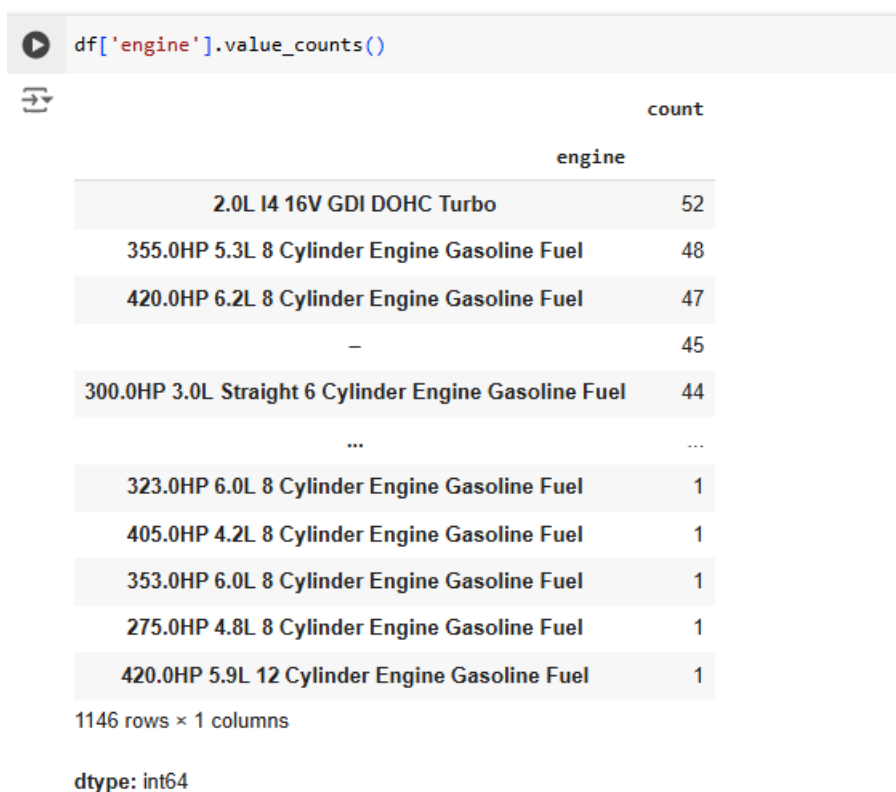


Figure 5: Value counts for engine column

After checking the engine column which has 1146 unique entries which will introduce complexity during encoding as well as inconsistencies within the data, I have decided to drop it entirely. The data quality is not good as some entries have less detail than others.

```
[7] df['fuel_type'].value_counts()
```



fuel_type	count
Gasoline	3309
Hybrid	194
E85 Flex Fuel	139
Diesel	116
–	45
Plug-In Hybrid	34
not supported	2

dtype: int64

Figure 6: Value counts for fuel type column

In the fuel type column, we run into our first column with missing values. An issue with this column is that majority of the data is in 'Gasoline' which may affect the models learning. After the initial test of the model, I can use undersampling to improve results.



```
# Count and display unique brands. Testing to see how i will pre-process the data.
unique_brands = df['brand'].unique()
print("Number of unique brands:", unique_brands)

# Show a sample of unique brands to see if one hot encoding will be suitable
print("Unique brands:", df['brand'].unique())
#[ :20]
```

Number of unique brands: 57
Unique brands: ['Ford' 'Hyundai' 'Lexus' 'INFINITI' 'Audi' 'Acura' 'BMW' 'Tesla' 'Land'
'Aston' 'Toyota' 'Lincoln' 'Jaguar' 'Mercedes-Benz' 'Dodge' 'Nissan'
'Genesis' 'Chevrolet' 'Kia' 'Jeep' 'Bentley' 'Honda' 'Lucid' 'MINI'
'Porsche' 'Hummer' 'Chrysler' 'Volvo' 'Cadillac' 'Lamborghini' 'Maserati'
'Volkswagen' 'Subaru' 'Rivian' 'GMC' 'RAM' 'Alfa' 'Ferrari' 'Scion'
'Mitsubishi' 'Mazda' 'Saturn' 'Bugatti' 'Polestar' 'Rolls-Royce'
'McLaren' 'Buick' 'Lotus' 'Pontiac' 'FIAT' 'Karma' 'Saab' 'Mercury'
'Plymouth' 'smart' 'Maybach' 'Suzuki']

Figure 7: Displaying unique brands

In the 'brand' column there are 57 different brands. Considering that this is real life data and certain brands generally cost more than others we cannot use label encoding as it will increase cardinality and make things difficult for the model. High cardinality means that there are many unique columns. I plan to group the brands into different categories for simplicity.

```
# Checking the unique colors in the dataset
unique_colors = df['int_col'].unique()
print("Unique colors:", unique_colors)
print("Number of unique colors:", len(unique_colors))
```

Unique colors: ['Black' 'Gray' 'Ebony.' 'Green' 'Pimento / Ebony' 'Brown' 'White' '-'
'Beige' 'Jet Black' 'Light Gray' 'Anthracite' 'Red' 'Nero Ade' 'Blue'
'Charcoal' 'Rioja Red' 'Canberra Beige/Black' 'Roast' 'Ivory / Ebony'
'Silk Beige/Black' 'Medium Pewter' 'Ice' 'Black / Express Red'
'Obsidian Black' 'Ebony' 'Orange' 'Caramel' 'Orchid' 'Graphite'
'Tan/Ebony' 'Cappuccino' 'Nero' 'Sahara Tan' 'BLACK' 'Hotspur' 'Blk'
'Black Onyx' 'Nougat Brown' 'Titan Black' 'Oyster W/Contrast' 'Silver'
'Macchiato Beige/Black' 'Sand Beige' 'Navy Pier' 'WHITE' 'Parchment.'
'Sarder Brown' 'Grace White' 'Cloud' 'Espresso' 'Mesa' 'Whisper Beige'
'Medium Dark Slate' 'Gold' 'Camel Leather' 'Tupelo' 'Black/Red'
'Parchment' 'Beluga Hide' 'Charles Blue' 'Global Black'
'Titan Black / Quarzit' 'Mocha' 'Portland' 'Medium Light Camel'
'Black / Saddle' 'Ash' 'Ebony / Pimento' 'Tan' 'Mistral Gray / Raven'
'Graystone' 'Satin Black' 'Sandstone' 'Medium Stone' 'ORANGE'
'Pimento Red w/Ebony' 'Macchiato' 'Ebony Black' 'Pearl Beige'
'Sakhir Orange' 'Kyalami Orange' 'Almond Beige' 'Medium Earth Gray'
'Diesel Gray / Black' 'Boulder' 'Walnut' 'Charcoal Black' 'Dark Auburn'
'Tan/Ebony/Ebony' 'Amber' 'Silk Beige/Espresso Brown' 'Black / Brown'
'Sakhir Orange/Black' 'Black / Stone Grey' 'AMG Black' 'Slate' 'BEIGE'
'Canberra Beige' 'Magma Red' 'Saiga Beige' 'Saddle Brown'
'Light Titanium' 'Dark Gray' 'Platinum' 'Ebony/Light Oyster Stitch'
'Shale' 'Red/Black' 'Agave Green' 'Cobalt Blue' 'Deep Cypress' 'Ceramic'
'Camel' 'Rhapsody Blue' 'Light Platinum / Jet Black' 'Shara Beige'
'Dark Galvanized' 'Giallo Taurus / Nero Ade' 'Yellow'
'Black w/Red Stitching' 'Black/Saddle Brown' 'Light Slate'
'Black/Gun Metal' 'White / Brown' 'Brandy' 'Mountain Brown' 'Linen'
'Porpoise' 'Aragon Brown' 'Oyster/Black' 'Gideon' 'Chestnut'
'Red / Black' 'Bianco Polar' 'Deep Garnet' 'Dark Ash'
'Ebony / Ebony Accents' 'Rock Gray' 'Cocoa / Dune' 'Beluga'
'Carbon Black' 'Sport' 'Medium Ash Gray' 'Black/Graphite'
'Macchiato/Magmagrey' 'Classic Red' 'Chateau' 'Graphite w/Gun Metal'
'Tension' 'Adrenaline Red' 'Hotspur Hide' 'Gray w/Blue Bolsters'
'Very Light Cashmere' 'Black / Gray' 'Deep Chestnut' 'Black / Graphite']
Number of unique colors: 156

Figure 8: Checking unique values in colour column

Similarly, I checked the colours columns in the dataset. The figure above shows that there is 156 interior colours. There are too many different colours in this dataset, and I will drop this column. Colours can affect car prices in real life as some are more desirable than others, but it will only have a small impact on price. Extracting the data from this column will cost a lot of time for little improvement in the model.

```
'designo Diamond White Bright' 'Iridium Silver Metallic'  

'Mango Tango Pearlcoat' 'GT SILVER' 'Crystal Black Pearl'  

'Eminent White Pearl' 'Lunar Blue Metallic' 'Magnetic Gray Metallic'  

'Arctic White' 'Silver Flare Metallic' 'Emin White' 'Custom Color'  

'Ember Pearlcoat' 'Twilight Blue Metallic' 'Burnished Bronze Metallic'  

'Ice Silver Metallic' 'Magnetite Gray Metallic' 'Jupiter Red'  

'Midnight Silver Metallic' 'Hellayella Clearcoat' 'Ironman Silver'  

'Alta White' 'Python Green' 'China Blue' 'Portofino Gray'  

'White Knuckle Clearcoat' 'Nautical Blue Pearl' 'Silician Yellow'  

'Arancio Borealis' 'Rosso' 'Graphite Grey' 'Lizard Green' 'Super Black'  

'Kinetic Blue' 'Electric Blue Metallic' 'Glacier Blue Metallic'  

'Red Obsession' 'Platinum Gray Metallic' 'Remington Red Metallic'  

'Reflex Silver' 'Deep Blue Metallic' 'Glacier Silver Metallic'  

'Carpathian Grey' 'Nightfall Gray Metallic' 'Lunar Rock'  

'Quartzite Grey Metallic' 'C / C']  

Number of unique colors: 319
```

Figure 9: Checking unique values in colour column

Using the same code but changing the column used, I also got the exterior colours. There are even more in this column. I will drop both colour columns.

```
[16] df['clean_title'].value_counts()  
#drop this column entirely. not useful data?
```



	count
clean_title	
Yes	3413

dtype: int64

Figure 10: Clean title value count

Next, I will do a value count check on the clean title column. All inputs in this column are already 'yes' and the remaining is missing. I will likely drop this column as it no value will be provided to the model with this column. It also seems like an error when this dataset was published.

```
[11] df.isna().sum()
```



	0
brand	0
model	0
model_year	0
milage	0
fuel_type	170
engine	0
transmission	0
ext_col	0
int_col	0
accident	113
clean_title	596
price	0

dtype: int64

Figure 11: Checking for empty values

After that, I did a quick check of the missing values in my dataset. Fuel type has 170 missing entries, accident has 113 missing entries and clean title has 596.

```
[ ] sns.scatterplot(data=df, x='milage', y='price')
```

```
>> <Axes: xlabel='milage', ylabel='price'>
```

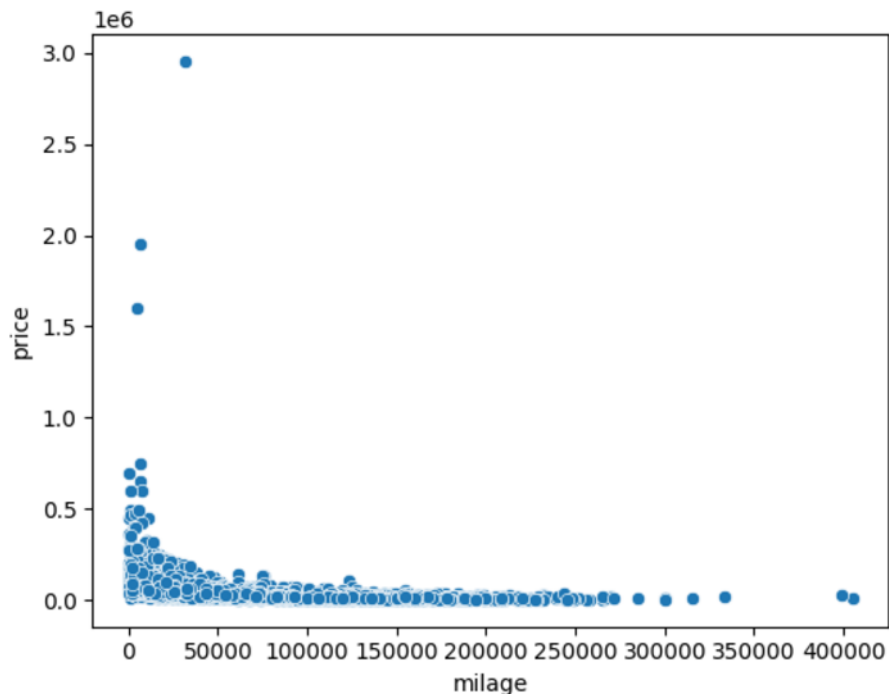


Figure 12: Scatterplot (Milage & Price)

The scatter plot in the figure above shows the relationship between milage and price. Cars with low milage tend to have higher prices. This is shown by the higher priced entries on the left side. As milage increases, it becomes less common to see cars priced at higher values. This is also known as negative correlation.

```
[ ] sns.scatterplot(data=df, x='model_year', y='price')
```

```
<Axes: xlabel='model_year', ylabel='price'>
```

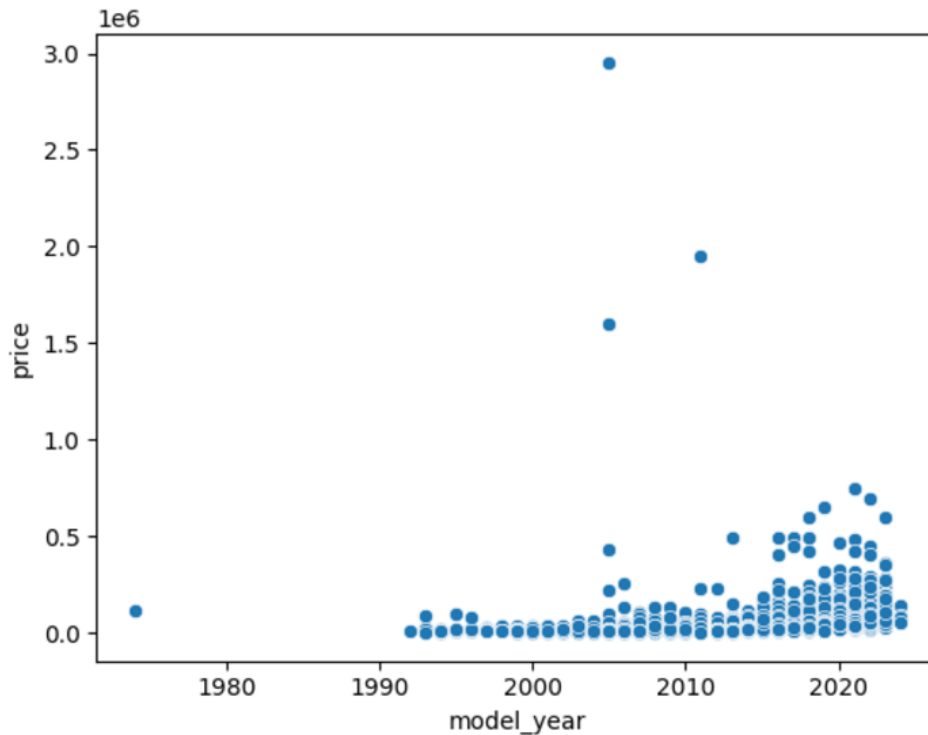


Figure 13: Scatterplot (Model Year & Price)

In the figure above, I created another scatter plot. This time showing the relationship between model year and the price. As shown, cars with a newer model are more likely to be priced higher although there are some older models priced higher than others. These entries are likely luxury or exotic brands. The general trend is that Cars from more recent years tend to have higher prices. This is also known as positive correlation.

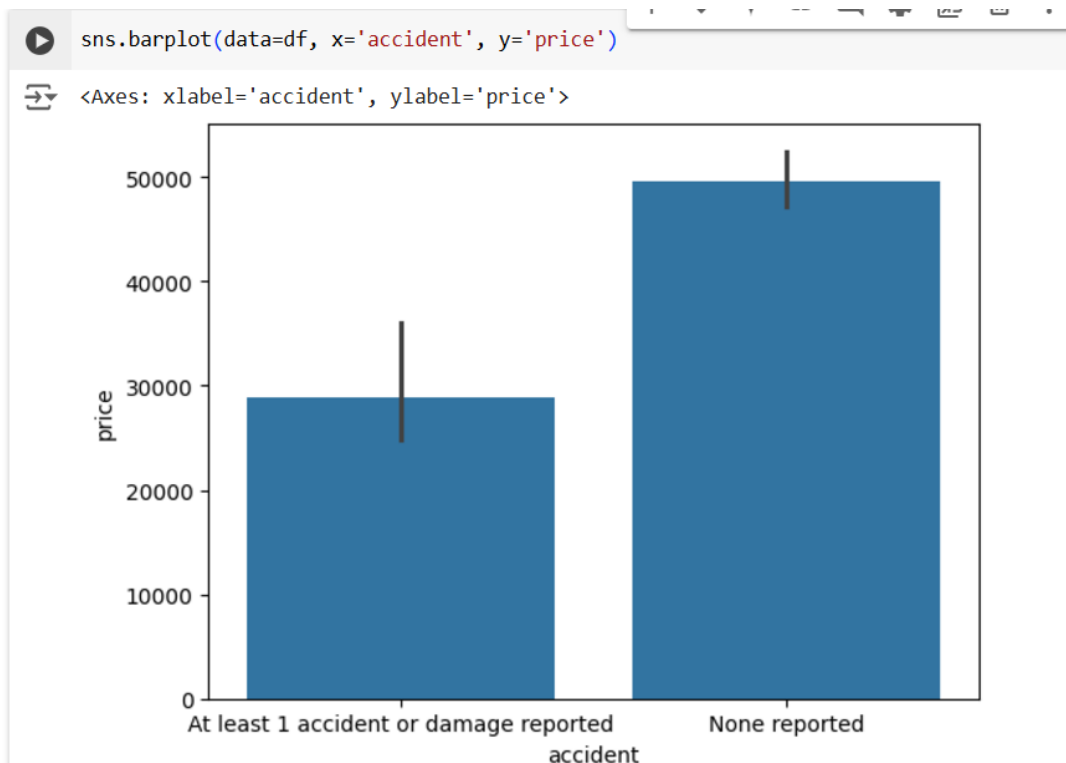


Figure 14: Accident column bar plot visualisation

Next, I created a bar plot to see the relationship between accident history and the price. As shown in the figure above, cars with no accidents reported are generally priced higher than cars that have been involved in accidents. This suggests that it is a significant factor when determining the price.

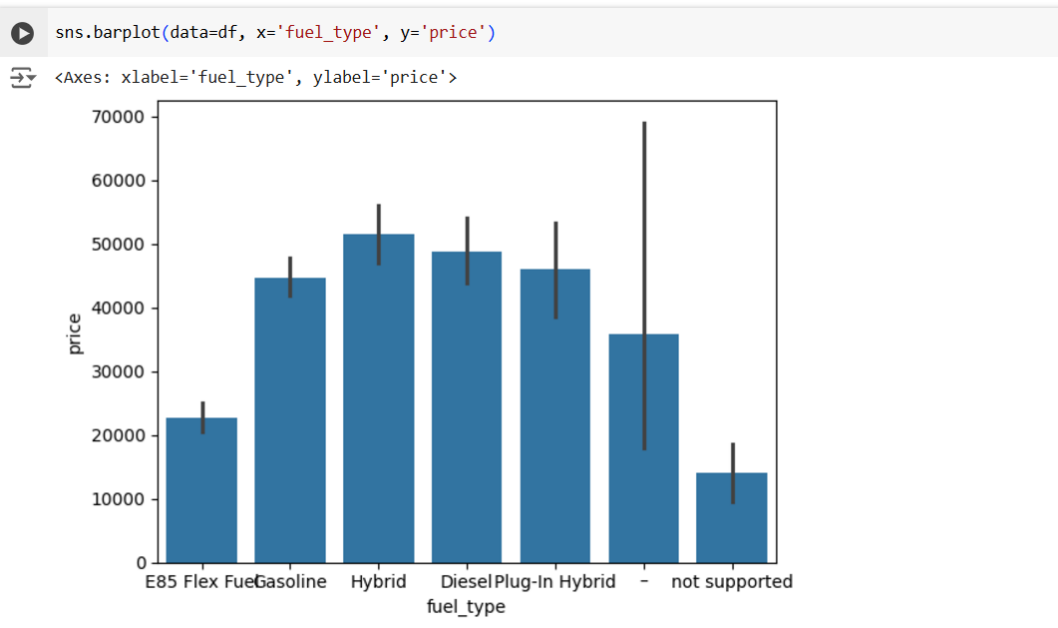


Figure 15: Fuel Type bar plot

The final visualisation in my EDA process is the bar plot for determining the relationship between price and fuel type. Hybrids are priced the highest followed closely by diesel. This is likely due to their efficiency or environmentally friendly features. This insight shows that it is another crucial feature for predicting prices.

2.4 EDA conclusions

In this section we have covered relationships between features such as milage, fuel type, model year, accident history compared to price. We have also identified missing values in certain columns such as accidents.

The main findings are the linear relationships with model year and milage. We also found out how cars with no accidents tend to sell for more and how different fuel types can increase the prices As well as filtering out columns which may not be as useful to our model.

After conducting EDA, I have been left with 7 features. Brand, Model year, Milage, fuel type, transmission, accident and price which is the target variable. The model will only have these features to train with.

3 Experimental Design

3.1 Identification of your chosen supervised learning algorithm(s)

Using the insights found during the EDA, I believe that using the random forest model to predict the car prices is a good choice due to its ability to handle categorical and continuous variables. This makes it a good choice for my dataset. Using this model also helps me reduce overfitting and it can operate with high accuracy even with some data missing. I have cut out a key column which was engine due to its data quality. It would be unnecessarily complex to encode it in a meaningful way. Using random forest could possibly help make up for that missing data. (h20.ai, n.d.)

Random forest is powerful and versatile tool that combines multiple decision trees and outputs the mean prediction of these individual trees. The key for success in this model is that individual trees can produce errors but majority of the group will be correct. However, there are drawbacks to this model. It can require a lot of memory which means it will take more computational power and time. But this is worth it as better results will be produced. (careerfoundry.com, 2020)

I will be comparing it to linear regression as these models have two different approaches. Linear regression assumes linear relationships between the target and feature variables, but it takes less time to model.

3.2 Identification of appropriate evaluation techniques

I will use Mean Squared Error (MSE) to evaluate the models' accuracy. A lower value indicates better performance, but this is sensitive to outliers, high priced brands like Lamborghini may impact the outputs. The impact of this is addressed through the usage of the Random Forest Model which can mitigate influence of outliers. (encord.com, n.d.)

Additionally, I will use R-Squared to provide a broader view of the models' performance. A value closer to 0 indicates a poor fit while a high value suggests that the model effectively captures relationships between features and the target.

Using Both metrics together gives me a more complete picture of the models' performance. MSE will assess prediction accuracy and R-Squared will reveal how much of the variance in car prices is explained by the models features.

3.3 Data cleaning and Pre-processing transformations

Data cleaning and preprocessing are important steps for machine learning models. Some benefits include ensuring data quality by fixing errors, missing values or inconsistencies that may impact the model. Good data preprocessing allows for more effective conclusions from the models. (Markus, 2024)

```
df['price'] = df['price'].str.replace('$', '').str.replace(',', '').astype(float)
df['price'].head()
```

	price
0	10300.0
1	38005.0
2	54598.0
3	15500.0
4	34999.0

dtype: float64

Figure 16: Price column data cleaning

I began by cleaning the price column, removing non-numerical characters such as dollar signs and then converting it into a float to allow performance of mathematical operations. This ensures the model can process the data without errors. Without this step, evaluation metrics such as MSE become unusable.

```
df['milage'] = df['milage'].str.replace('mi', '').str.replace(',', '').str.replace(' ', '').astype(float)
df['milage'].head()
```

	milage
0	51000.0
1	34742.0
2	22372.0
3	88900.0
4	9835.0

dtype: float64

Figure 17: Milage column data cleaning

Next, I did the same thing for the milage column. Removing commas, extra spaces and 'mi' which was in all the entries. Then converted it into a float so that it can be used for the models training.

```
[48] df.drop(columns=['clean_title'], inplace=True)
```

Figure 18: Dropping clean title column

As we discovered during the EDA, clean title does not offer any important data. All entries were labelled as yes or had missing data. I decided to drop the column completely as this would have little impact on the models outputs.

```
[138] df['accident'].value_counts()
```



accident		count
None reported		2910
At least 1 accident or damage reported		986

dtype: int64

```
[139] df['accident'].isna().sum()
```



113

Figure 19: Accident value count check and missing value check

Then I did a value counts check on the accident column for confirmation that there is only two possible values. I also checked for missing values using 'isna().sum()'. There are 113 missing values.

```
df['accident'] = df['accident'].replace('None reported', 0)
df['accident'] = df['accident'].replace('At least 1 accident or damage reported', 1)

df['accident'] = df['accident'].fillna(0) #imputing most common value
df['accident'] = df['accident'].astype(int)
```

Figure 20: Manual Encoding & Handling missing values

After confirming two possible values I used manual encoding to convert the column into binary. I replaced 'None reported' with 0, 'At least 1 accident or damage reported' with 1 to turn it into a fully binary column. Then I dealt with the missing values by imputing the most common value in the column which was '0' and finished off by converting the strings into integers to make it more model friendly.

Initially, I had used label encoding but it was labelling the column in a counter intuitive way. Cars with accidents were being labelled as '0' and cars with '1' meant no accident. This could cause confusion later when interpreting results. Manual encoding aligns better with general expectations of how the data should be structured and improves clarity.

```
[111] df['accident'].value_counts()
```

```
count
accident
0      3023
1       986

dtype: int64
```

```
[141] df['accident'].isna().sum()
```

```
0
```

Figure 21: Confirmation of encoding

```
[142] df = df.drop(columns=['ext_col', 'int_col', 'model', 'engine'])
```

```
[143] df.head()
```

```
brand  model_year  mileage  fuel_type  transmission  accident  price
0    Ford        2013   51000.0    E85 Flex Fuel      6-Speed A/T         1   10300.0
1  Hyundai        2021  34742.0    Gasoline        8-Speed Automatic         1   38005.0
2   Lexus        2022  22372.0    Gasoline        Automatic          0  54598.0
3 INFINITI        2015  88900.0    Hybrid         7-Speed A/T          0  15500.0
4   Audi        2021   9835.0    Gasoline        8-Speed Automatic          0  34999.0
```

Figure 22: Dropping columns

Following my discoveries during EDA, I dropped all the remaining columns which I have deemed redundant for the model. This leaves us with the features that we will be using to train and test the model. I did a quick check to see what was left.

```
[144] most_common_transmissions = df['transmission'].value_counts().head(5).index

def group_transmissions(tran):
    if tran in most_common_transmissions:
        return tran
    else:
        return 'Other'

df['transmission'] = df['transmission'].apply(group_transmissions)
```

Figure 23: Grouping transmissions

To encode the transmission column, I have identified the top 5 common transmissions in the dataset and grouped the remaining ones under a new category called 'Other'. This approach keeps the cardinality low while also retaining majority of the relevant information.

In the dataset there were transmissions which just did not have enough entries and would add extra noise for the model. By grouping less common categories together, potential noise that would affect the model is minimised. This could help improve the model's ability during training.

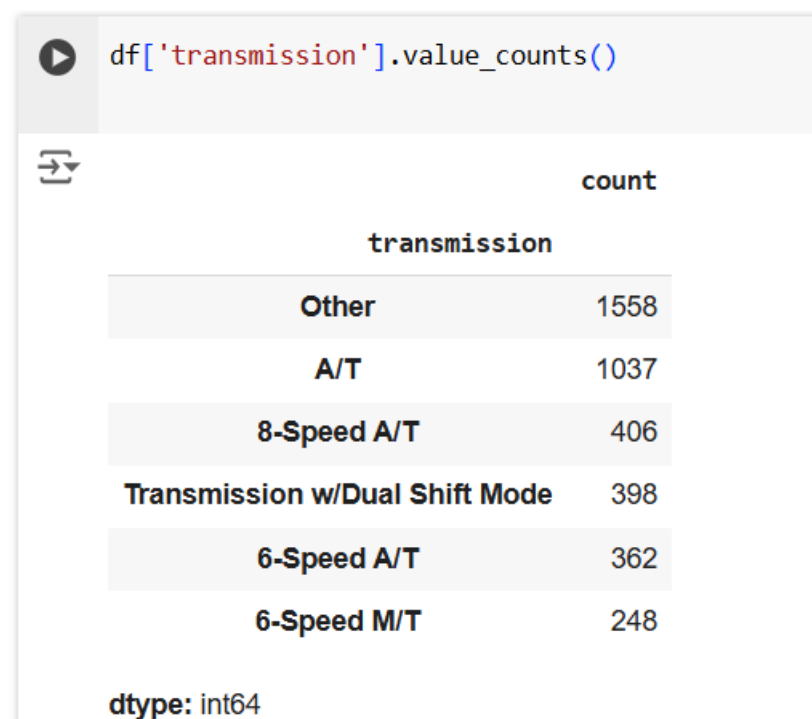


Figure 24: Transmission updated value count

As we can see from the figure above, majority of the data set has been captured.

```

df['transmission'] = df['transmission'].replace('A/T', 0)
df['transmission'] = df['transmission'].replace('8-Speed A/T', 1)
df['transmission'] = df['transmission'].replace('Transmission w/Dual Shift Mode', 2)
df['transmission'] = df['transmission'].replace('6-Speed A/T', 3)
df['transmission'] = df['transmission'].replace('6-Speed M/T', 4)
df['transmission'] = df['transmission'].replace('Other', 5)

```

Figure 25: Manual Encoding transmissions

Manual encoding was then conducted on this column using the replace function. The figure above shows the values associated with each number.

Initially, I did use label encoding but I was confused by the results and believed that they were counter intuitive. Label encoding assigns numeric labels in an arbitrary order which could lead to confusion when interpreting results. By manually encoding it, I have improved the clarity and useability.

```

[148] df['transmission'].value_counts()
      #successfully encoded!

```



count	
transmission	
5	1558
0	1037
1	406
2	398
3	362
4	248

dtype: int64

Figure 26: Encoded Value count check

The figure above shows that the column was successfully encoded.

```
[149] df['fuel_type'].value_counts()
```



fuel_type		count
Gasoline		3309
Hybrid		194
E85 Flex Fuel		139
Diesel		116
-		45
Plug-In Hybrid		34
not supported		2

dtype: int64

Figure 27: Fuel Type value count check

Moving onto the fuel column, I started off with a quick check of all the values to see which approach I would take.

```
[150] most_common_fueltypes = df['fuel_type'].value_counts().head(4).index
```

```
def group_fueltypes(type):  
    if type in most_common_fueltypes:  
        return type  
    else:  
        return 'Other'  
  
df['fuel_type'] = df['fuel_type'].apply(group_fueltypes)
```

Figure 28: Grouping fuel types

Similar to how I encoded the transmission column, I identified the most common 4 fuel types and created a new group for the remaining values. This also helped me deal with the missing values as they were grouped as 'Other'. This reduces the complexity of the column but also retracts the useful information.

By grouping the less common fuel types together, I reduced noise in the dataset which could allow the model to figure out prevalent patterns which could give me better results.

```
[152] df['fuel_type'] = df['fuel_type'].replace('Gasoline', 0)
df['fuel_type'] = df['fuel_type'].replace('Hybrid', 1)
df['fuel_type'] = df['fuel_type'].replace('E85 Flex Fuel', 2)
df['fuel_type'] = df['fuel_type'].replace('Diesel', 3)
df['fuel_type'] = df['fuel_type'].replace('Other', 4)
```

Figure 29: Manual Encoding fuel types

I then used manual encoding to convert them into numerical values so that they can be compatible with the model. The mapping is shown below:

<u>Fuel Type</u>	<u>Encoded As:</u>
Gasoline	0
Hybrid	1
E85 Flex Fuel	2
Diesel	3
Other	4

Figure 30: Encoded Mappings for fuel type

By using manual encoding I ensured that the data is consistent and has increased clarity which makes it easier for the algorithm to understand the relationship between fuel types and the target variable.

```
[154] Exotic_brands = ('Aston', 'Bentley', 'Ferrari', 'Lamborghini', 'Rolls-Royce', 'Bugatti', 'McLaren')
Luxury_brands = ('Audi', 'BMW', 'Lexus', 'Mercedes-Benz', 'Porsche', 'Tesla', 'Jaguar', 'Land', 'Maserati', 'Genesis')
MidRange_brands = ('Acura', 'Lincoln', 'Volvo', 'MINI', 'Cadillac', 'Subaru', 'GMC', 'RAM', 'Volkswagen', 'Alfa')
Economy_brands = ('Ford', 'Hyundai', 'Toyota', 'Honda', 'Nissan', 'Chevrolet', 'Kia', 'Jeep', 'Dodge', 'Mazda', 'Mitsubishi', 'Chrysler', 'Buick', 'Saturn', 'Pontiac', 'FIAT', 'Suzuki')
Other_brands = ('Lucid', 'Hummer', 'Rivian', 'Scion', 'Polestar', 'Lotus', 'Mercury', 'Plymouth', 'Saab', 'Maybach', 'smart', 'Karma')

def group_brands(brand):
    if brand in Exotic_brands:
        return '1' #1 = exotic
    elif brand in Luxury_brands:
        return '2' #2 = luxury
    elif brand in MidRange_brands:
        return '3' #3 = mid range
    elif brand in Economy_brands:
        return '4' #4= economy
    else:
        return '5' #5 = other brands. these brands are not very common or less known brands.

df['brand'] = df['brand'].apply(group_brands)
```

Figure 31: Grouping Brands

To encode the brand column, I have grouped brands based on their real-life market segments. This approach reduces the high cardinality of the brand column and makes the dataset much more manageable for the model.

I have grouped them into categories as follows:

Category	Description	Encoded As:
Exotic brands	High end luxury sports cars such as Aston martin or Lamborghini	1
Luxury brands	Premium cars that provide performance and comfort such as BMW or Audi	2
Mid-range brands	These cars Offer a balance between affordability and luxury such as Volvo	3
Economy brands	Mostly known for being affordable	4
Other brands	less common brands such as Hummer	5

Figure 32: Brand group specifications

I grouped them by applying a custom function which checks if a given brand falls into which category. Then it returns a numerical encoding for that category. Reducing the number of unique categories in the column allows the model to focus on patterns rather than overfitting to specific brands. There are not enough entries for each brand for it to be unique.

Initially I did only plan to have luxury, mid range and economy but while I was grouping them I thought it was more suitable to introduce exotic and 'Other' categories. This approach helps reflects the diversity better and less frequent brands do not introduce noise into the model.

```
[155] df['brand'] = df['brand'].astype(int)
```

Figure 33: Converting brand column into integer

To complete the encoding of the brand column, I converted the values into an integer to make it compatible with the model.

```
[156] df.info()
```

```
>>> <class 'pandas.core.frame.DataFrame'>
RangeIndex: 4009 entries, 0 to 4008
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   brand           4009 non-null   int64
 1   model_year      4009 non-null   int64
 2   milage          4009 non-null   float64
 3   fuel_type       4009 non-null   int64
 4   transmission    4009 non-null   int64
 5   accident        4009 non-null   int64
 6   price           4009 non-null   float64
dtypes: float64(2), int64(5)
memory usage: 219.4 KB
```

Figure 34: information check

Using 'df.info()' I did a final check on the dataset before moving onto the modelling. There are no null values and all data types are integer or float ensuring compatibility with the model.

3.4 Limitations and Options

Removing key features such as engine or colours could impact the model's performance as it is important information. However, keeping these would have made the model very complex. Furthermore, manual encoding which was used could have oversimplified the data and imputing data in the accident column could have introduced inaccuracies within the data.

4 Predictive Modelling / Model Development

4.1 The predictive modelling process

```
[50] X = df.drop('price', axis=1)
     y = df['price']
```

```
[51] X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Figure 35: Train-Test-Split Setup

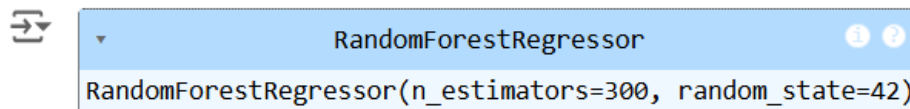
To prepare the dataset for modelling, I created x and y sets. Y was the target variable and X was the features. I then used train-test-split using an 80-20 ratio for a fair evaluation.

```
[52] sc = StandardScaler()
     X_train = sc.fit_transform(X_train)
     X_test = sc.transform(X_test)
```

Figure 36: Scaling data

The features were then scaled to ensure numerical stability for the model. Scaling is important because it can help address outliers, balance feature influences and improve algorithm behaviour. (Shivanipickl, 2023)

```
[76] rf_model = RandomForestRegressor(n_estimators=300, random_state=42)
      rf_model.fit(X_train, y_train)
```



```
[77] y_pred = rf_model.predict(X_test)
```

```
[78] y_train_pred = rf_model.predict(X_train)
```

Figure 37: Random Forest Model

My chosen model is the random forest regressor because of its ability to handle non-linear relationships. The model uses 300 'n_estimators' which improves the stability of the predictions as there is more trees for the model to use. However, it may take more computational resources. Besides that, the default parameters have been used for the initial test. Random forest performs well even with minimal tuning as it is a robust model. This means that I can have a strong performance right away and tune it as we see results. I have used a random state of 42 to ensure reproducibility of the results. In the figure above I have initialised the test predictions and the training predictions.

Tuning options such as min_samples_leaf, max_depth, bootstrap were all left as default. Max depth is set to none which means trees are grown until they have less than the minimum required to split. Min_samples_split is set to 2 at default. This means that it needs two samples to split an internal node. These are just some of the options. (scikit-learn, 2024)

4.2 Evaluation results on “seen” data

```
[79] #seen data
     mse = mean_squared_error(y_train, y_train_pred)
     r2 = r2_score(y_train, y_train_pred)
     print("Training MSE:", mse)
     print("Trainig R squared:", r2)
```

```
⇒ Training MSE: 166155613.07746366
   Trainig R squared: 0.9366080768387286
```

Figure 38: Training Data Scores (Random Forest)

Once the model development was completed, I began the evaluation process by assessing the model's performance on training data. As shown in the figure above, the model performed exceptionally well on the training data. From the R-squared score we can see it was able to explain 93% of the variance.

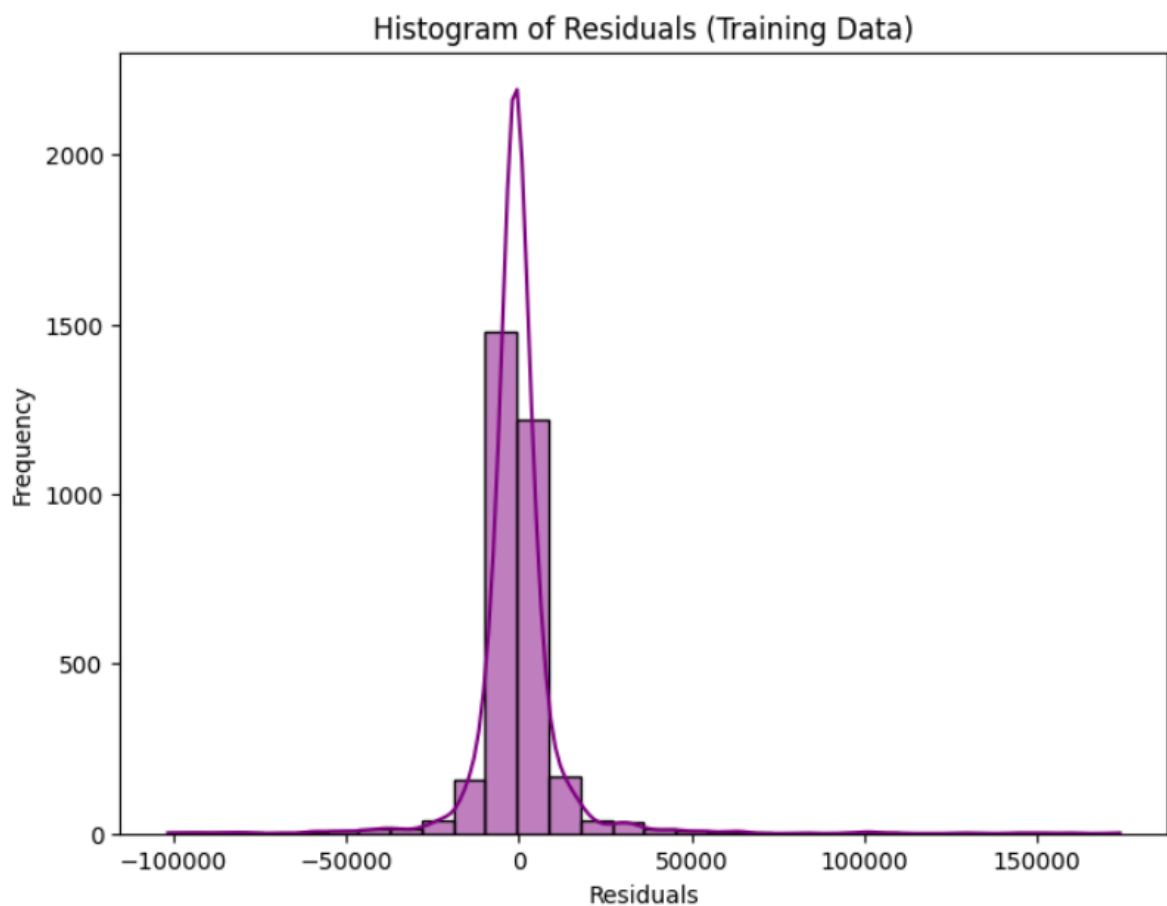


Figure 39: Residuals Histogram (Random Forest)

This histogram above shows the residuals of the model during the training data. The sharp peak shows that the model is struggling with some car predictions which could be due to unobserved variables or noisy data. The model performs well on training data but would need tuning to handle outliers better.

5 Evaluation and further modelling improvements

Step 1: Model Selection

Car price prediction is a regression problem. Models used are Random Forest and Linear Regression.

Step 2: Data Preparation

As mentioned previously, I removed columns that would introduce too much complexity such as colours and engine. Data cleaning was also completed to make columns such as price and milage compatible with models. I then completed data pre-processing by using techniques such as mode imputation to handle missing values, encoding variables such as fuel type and accident using manual encoding. Manual encoding gives me more control over the data and makes it more readable. Features were also scaled to ensure that the data is compatible with multiple models. Overall, by completing these steps the dataset becomes well structured, with clean and encoded features ready for model training. The simplicity of it could also help reduce noise within the data.

Step 3: Model Development

I have selected the random forest model due to its ability to handle complex relationships. This method combines multiple decision trees which helps improve accuracy and handling complex data. I split my data into Training (80%) and testing (20%) which allows the model to learn from a variety of data while being tested on the unseen data.

To train the model, I used the default parameters but I did tune the 'n_estimators' to 300 for a better insight and used a random state of 42 so that the results could be reproduced. This would give me an initial baseline and give insights on what I could do next to improve it. Hyperparameter tuning was not performed besides what is mentioned for the initial evaluation.

Step 4: Model Evaluation

To assess the performance of the random forest regressor, I used R squared and Mean squared error. R squared tells us the variance in the target variable that is explained by the model. A value closer to one means better performance. MSE measures the average squared difference between predicted and actual values. For the testing data, the model performed well as discussed earlier. An R^2 score of 0.936 which shows it could capture the complex relationships between features and the target variable. The MSE was also much lower than on unseen data.

```
[80] #test data
     mse = mean_squared_error(y_test, y_pred)
     r2 = r2_score(y_test, y_pred)
     print("MSE:", mse)
     print("R squared:", r2)
```

```
→ MSE: 17585337646.853184
   R squared: 0.13964252937259924
```

Figure 40: Test Data Scores (Random Forest)

As shown in the figure above, the test data shows that the model struggled with unseen data. The R^2 score is much closer to zero and the MSE is a lot higher. This shows that the model is overfitting and struggles to generalise to new data. This suggests that further tuning and optimisation is needed. I can explore techniques such as cross validation and hyperparameter tuning. Changing the max depth could help the model adapt.

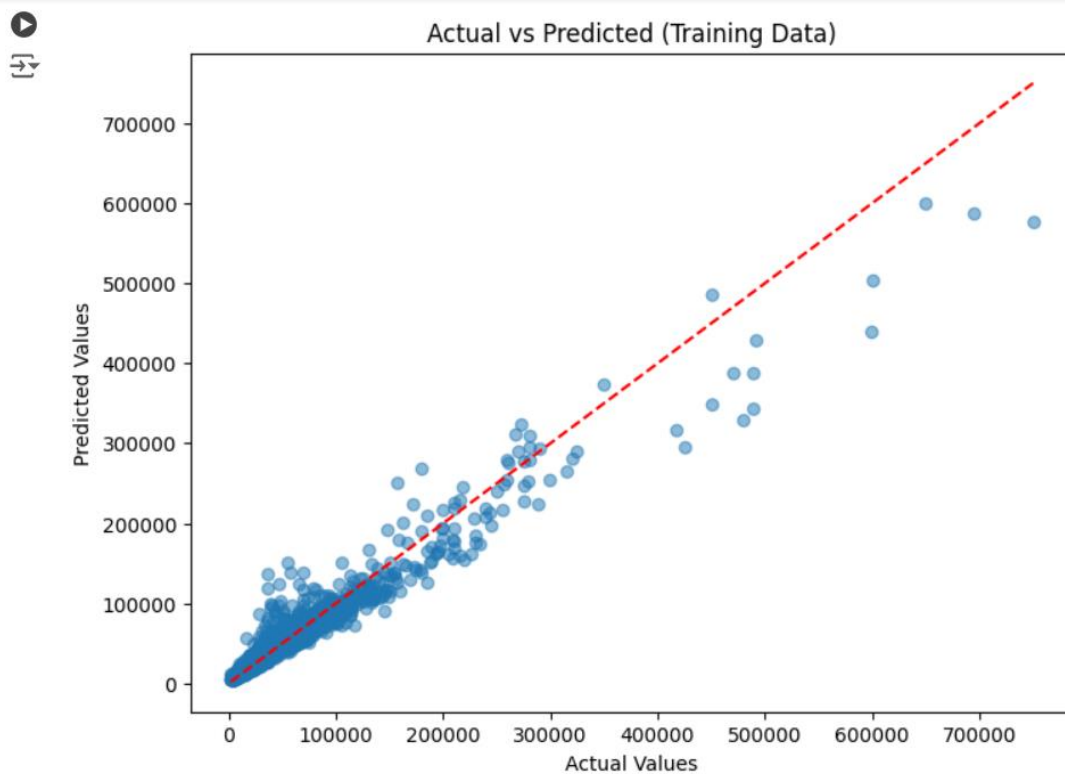


Figure 41: Random Forest Prediction scatterplot

The scatterplot above shows that the model is performing well as many of the dots are close to the straight line. The tightness implies that the model is confident in its predictions for majority of the data. However there are some that are quite far off, possibly due to unusual data.

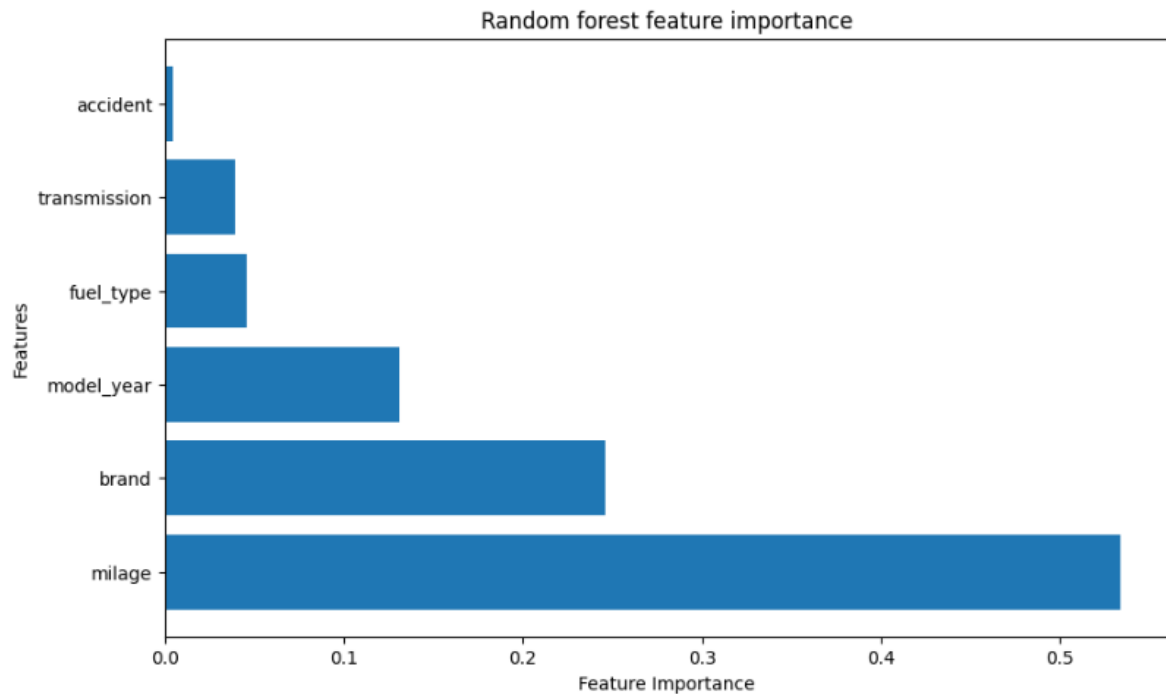


Figure 42: Random Forest Feature Importance

The figure above shows that the most important features used for prediction were milage, brand and the model year. This shows that low milage, newer models and luxury brands are the biggest price influences. On the other hand, accident history, fuel type and transmission don't seem to hold as much significance. This is very close to what is expected and close to real world data which shows that the model was able to capture the relationships between the features and the target variable effectively. This enhances the interpretability of the results.

Step 5: Model Improvement

```
[147] # Define the parameter grid
      param_grid = {
          'n_estimators': [100, 300, 500],
          'max_depth': [10, 15, None],
          'min_samples_split': [2, 5, 10],
          'min_samples_leaf': [1, 5, 10]
      }

[69] # Instantiate the GridSearchCV
      grid_search = GridSearchCV(estimator=RandomForestRegressor(random_state=42),
                                param_grid=param_grid,
                                cv=5, # 5-Fold Cross-Validation
                                scoring='r2',
                                n_jobs=-1)

[70] # Fit the grid search model
      grid_search.fit(X_train, y_train)
```

Figure 43: Grid Search (Random Forest)

The model was not performing well on unseen data which means that hyperparameter tuning will be required to improve results. Scaling was already done to the data which did slightly improve its performance. To find out the best tuning settings for the model I decided to use a grid search. This would check through all the options and then provide me with the best ones.

```
[70] # Fit the grid search model
      grid_search.fit(X_train, y_train)
```

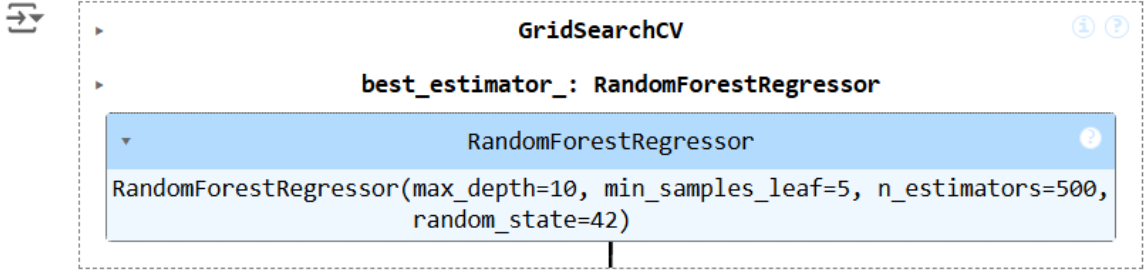


Figure 44: Best Tuning Options returned by GridSearch (Random Forest)

The grid search took 10 minutes and returned the following:

The `n_estimators` were increased to 500 to help the model capture more complex patterns. The max depth was set to 10 to prevent overfitting by avoiding overly large trees. Minimum samples split was set to 2 to allow the model to learn from more patterns but only from meaningful splits. Lastly, the minimum samples leaf was set to 5 to ensure that each leaf has enough data to avoid overfitting to smaller subsets such as the exotic category.

```
[92] improved_linearreg_model = LinearRegression(positive = True)
```

```
[93] kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

Figure 45: Model Tuning (Linear Regression)

Linear regression has no hyperparameters because it uses the OLS method (Chawla, 2023). However, I did attempt to tune it by forcing positive predictions only.

Step 6: Validation

```
[247] Improved_rf_model = RandomForestRegressor(n_estimators=500, max_depth=10, min_samples_split=2, min_samples_leaf=5, random_state=42)
```

```
[237] kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
[242] cv_results = cross_val_score(Improved_rf_model, X_train, y_train, cv=kf, scoring='r2')
```

```
print("Cross-Validation R2 Scores:", cv_results)
print("Mean R2 Score:", np.mean(cv_results))
```

```
Cross-Validation R2 Scores: [0.46489878 0.60893032 0.56337898 0.54477497 0.68646918]
Mean R2 Score: 0.5736904461199204
```

Figure 46: Updated Model & Cross Validation Scores (Random Forest)

After applying the hyperparameter tuning, I used kfold cross validation to get a robust assessment on the models performance. The Mean R^2 score is 0.57 which I think is good especially considering the simplicity of the data. I used 5 folds to get a good estimate of how it would perform while also not needing much computational power.

```
[94] linearreg_cv_results = cross_val_score(improved_linearreg_model, X_train, y_train, cv=kf, scoring='r2')
```

```
[97] print("Cross-Validation R2 Scores:", linearreg_cv_results)
print("Mean R2 Score:", np.mean(linearreg_cv_results))
```

```
Cross-Validation R2 Scores: [0.13276944 0.14706338 0.11103305 0.10497532 0.14437886]
Mean R2 Score: 0.12804400859850645
```

Figure 47: Updated Model & Cross Validation Scores (Linear Regression)

I also evaluated this model using kfold cross validation. I had used 10 splits instead of 5 for a better evaluation. Some splits did perform better than the base model while others had worse results. Overall, this model seems unfit for this type of problem likely due to the complex relationships. More advanced models would be better suited for this task.

To further improve these results, I could conduct feature engineering. I did try to implement 'car_Age' feature but decided that it would be redundant as the model year is already there and it would add more complexity which is what I want to avoid.

Step 7: Comparative Analysis of Multiple Algorithms

The two models I have developed are linear regression and random forest, chosen for their contrasting approaches. Random forest is a non-linear model that averages predictions from multiple decision trees that are trained on different subsets of features, which allows it to capture complex relationships and highlight feature importance. However, it is computationally expensive and difficult to interpret due to the number of trees. (Yadav, 2024)

Linear regression assumes a linear relationship between features and the target making it fast and easy to interpret. It is also quicker to train. However, it is sensitive to outliers and is prone to overfitting. (Yadav, 2024)

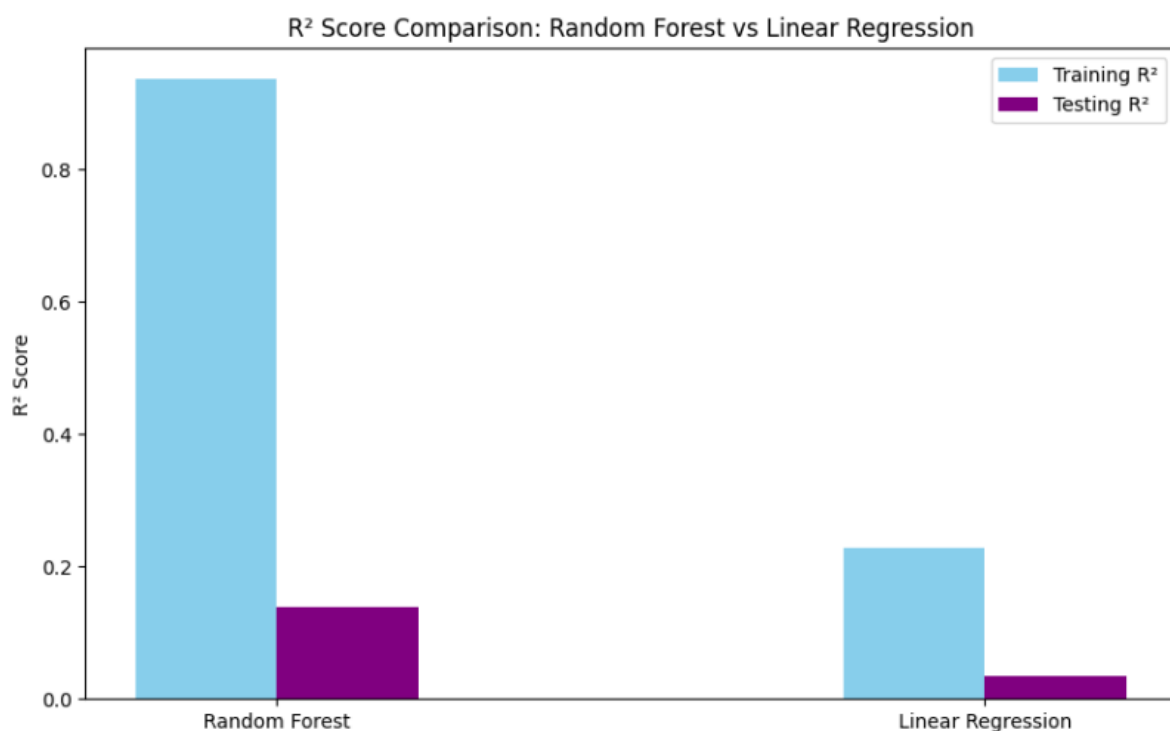


Figure 48: R² Comparison of initial models

The figure above shows the R² scores for both models (before tuning) during testing and training.

Random forest excels at this task as it can capture the complex relationships evident through the high R² scores. But the model is overfitting which is shown through the high R² score of 0.93 and the lower test score of 0.13.

However, linear regression is struggling due to its assumption of linear features. This model is underfitting which means it is not performing well at all which is shown through the low training (0.22) and low testing (0.035) R² scores.

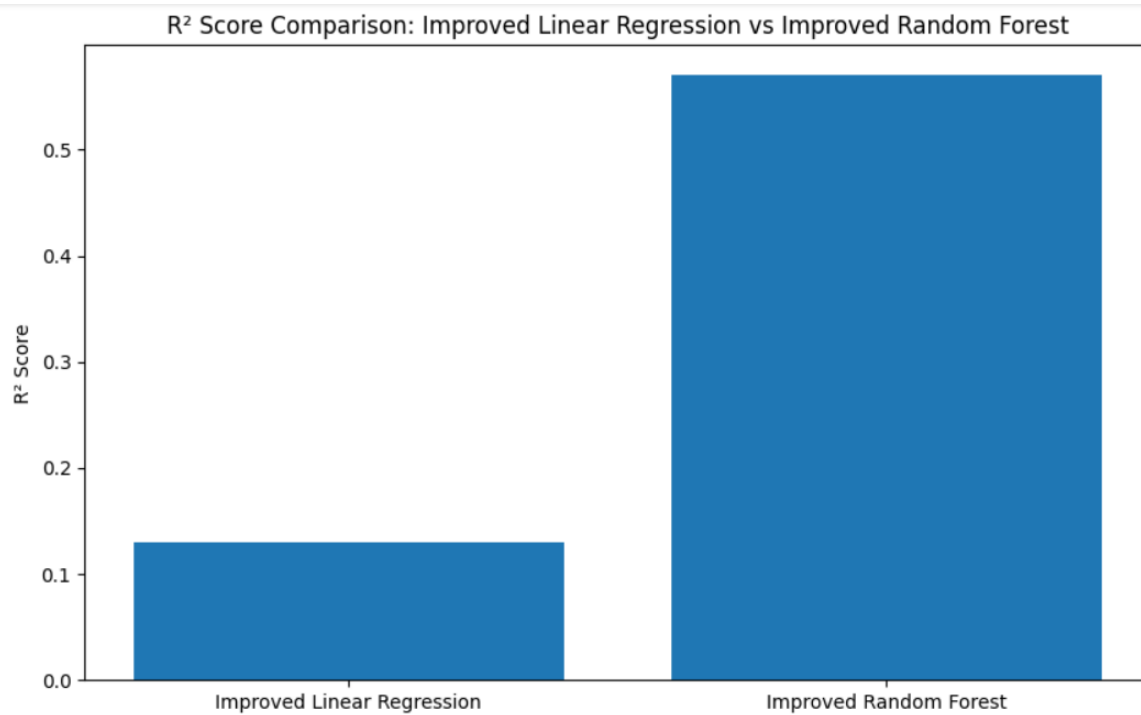


Figure 49: Updated Model R^2 Score Comparisons

After tuning both models, I used K-fold cross validation to compare them. Linear regression scored 0.13 while random forest scored 0.57. This highlights the flaws of linear regression and its inability to capture complex relationships. The results were still the same with random forest proving to be much better than linear regression.

6 Conclusion

6.1 Summary of results

In this project, I aimed to develop a machine learning model that predicts car prices using various features. The key steps included, EDA, pre-processing, visualization techniques, handling outliers and comparing models.

```
#removing potential outliers. The category 5 is labelled as 'Other' which could contain data that confuses the model.
df_filtered = df[df['brand'] != 5]

# Model Development -
X_filtered = df_filtered.drop('price', axis=1)
y_filtered = df_filtered['price']

X_train, X_test, y_train, y_test = train_test_split(X_filtered, y_filtered, test_size=0.2, random_state=42)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Figure 50: Final Model Development & Data filtering

The final model was a Random Forest regressor. Considering the overfitting issue, I tuned hyperparameters and modified data to reduce noise and improve predictions. I removed the 'Other' category to filter out data which likely confused the model. For a fair comparison against the base model, I used train test split and scaled the data.

```
Improved_rf_model = RandomForestRegressor(n_estimators=500, max_depth=10, min_samples_split=10, min_samples_leaf=10, random_state=42)
Improved_rf_model.fit(X_train, y_train)

y_pred = Improved_rf_model.predict(X_test)
y_train_pred = Improved_rf_model.predict(X_train)
```

Figure 51: Improved Model Hyperparameters

Using the results from the GridSearch, I tuned the model accordingly. I also experimented with the options for better results. I increased the min_samples_split from 2 to 10 to allow the model to generalise better and avoid overly specific patterns.

```
# Model Evaluation
mse_train = mean_squared_error(y_train, y_train_pred)
r2_train = r2_score(y_train, y_train_pred)
mse_test = mean_squared_error(y_test, y_pred)
r2_test = r2_score(y_test, y_pred)

print("Training MSE:", mse_train)
print("Training R²:", r2_train)
print("Test MSE:", mse_test)
print("Test R²:", r2_test)
```

```
↔ Training MSE: 3711671876.794532
   Training R²: 0.321409535387161
   Test MSE: 4685277028.678787
   Test R²: 0.3532894533396125
```

Figure 53: Scores for final random forest model

The performance of both models are as follows:

Model	Training R ²	Testing R ²
Initial Random Forest	0.93	0.13
Final Random Forest Model	0.32	0.35

Figure 54: Initial Random Forest VS Final Random Forest

The initial model excelled during training but failed to generalise on test data. After tuning and data filtering, the final model was able to generalise better despite a drop in testing performance. This demonstrates the importance of reducing overfitting and handling noisy data when building predictive, robust models.

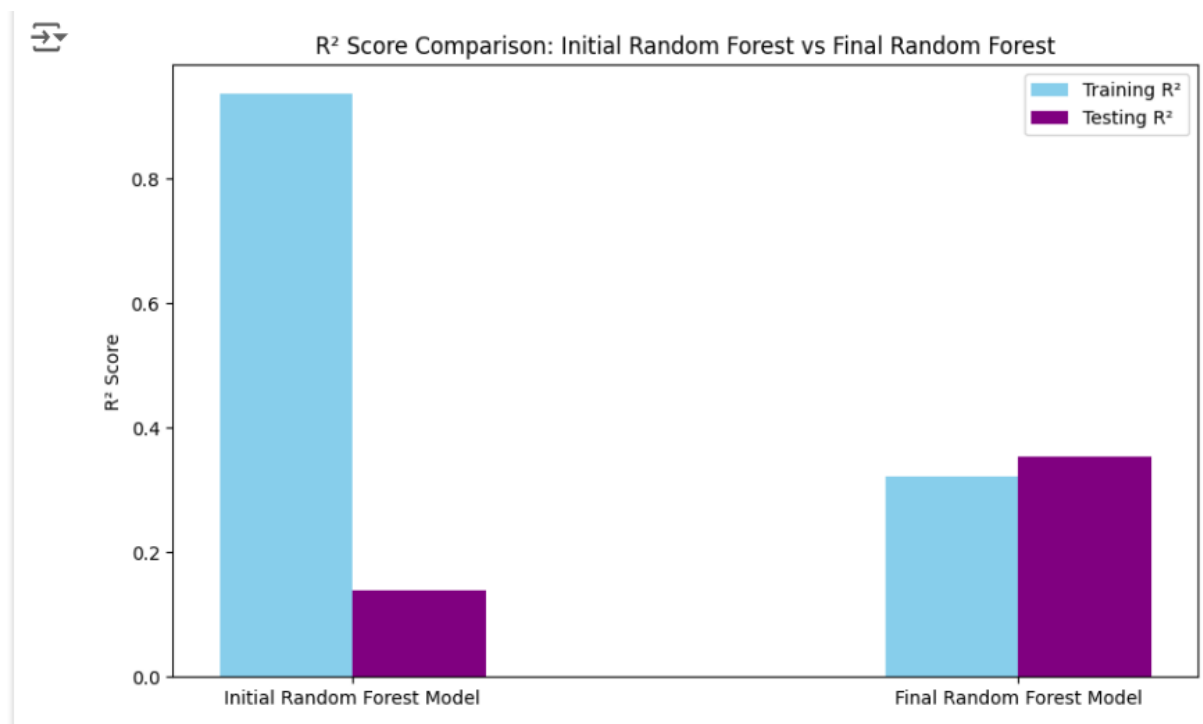


Figure 55: Initial Random Forest VS Final Random Forest Bar Plot Visualisation

As shown in the figure above, the final models training performance was decreased however this led to significantly better generalisation on unseen data. This highlights the importance of reducing overfitting and handling noisy data.

In conclusion, the base model was overfitting the training data while the final tuned model was giving a more balanced performance through tuning and data filtering.

6.2 Reflection on Individual Learning

During this coursework I learned about the full life cycle of building predictive models including EDA, pre-processing, model development and hyperparameter tuning. I also learned the importance of data quality.

I particularly enjoyed the pre-processing stage as it allowed me to transform the data into something clean ready for modelling. I also found tuning the models rewarding. Exploring different parameters and how they influenced the results was interesting.

To improve my skills further, I would learn data sampling techniques for even better results, more advanced models that may have done better with this same problem. I would also like to review my coding skills as I encountered challenges often and had to refer to materials such as the labs quite often. I could do this through online tutorials and resources such as FreeCodeCamp.

7 References

h2o.ai. (n.d.). *How Is Random Forest Used For Classification and Regression Problems?* [online] Available at: <https://h2o.ai/wiki/random-forest/>.

encord.com. (n.d.). *Mean Square Error (MSE) | Machine Learning Glossary | Encord | Encord*. [online] Available at: <https://encord.com/glossary/mean-square-error-mse/>.

careerfoundry.com. (2020). *What is Random Forest? [Beginner's Guide + Examples]*. [online] Available at: <https://careerfoundry.com/en/blog/data-analytics/what-is-random-forest/#what-are-the-disadvantages-of-random-forest>.

Markus, G. (2024). *Why Data Preprocessing is Necessary in Data Science*. [online] Medium. Available at: <https://medium.com/@gideonmarkus/why-data-preprocessing-is-necessary-in-data-science-546235345fdb>.

Shivanipickl (2023). *What is Feature Scaling and Why Does Machine Learning Need It?* [online] Medium. Available at: <https://medium.com/@shivanipickl/what-is-feature-scaling-and-why-does-machine-learning-need-it-104eedebb1c9>.

Chawla, A. (2023). *Why Sklearn's Linear Regression Has No Hyperparameters?* [online] Dailydoseofds.com. Available at: <https://blog.dailydoseofds.com/p/why-sklearns-linear-regression-has>.

scikit-learn. (2024). *RandomForestRegressor*. [online] Available at: <https://scikit-learn.org/1.5/modules/generated/sklearn.ensemble.RandomForestRegressor.html>.

Yadav, A. (2024). *Linear Regression vs Random Forest - Amit Yadav - Medium*. [online] Medium. Available at: <https://medium.com/@amit25173/linear-regression-vs-random-forest-7288522be3aa>.

7.1 Appendix

```
sns.barplot(data=df, x='transmission', y='price')  
#too many different transmissions make the plot unreadable. But some are definitely more expensive than others.  
<Axes: xlabel='transmission', ylabel='price'>
```

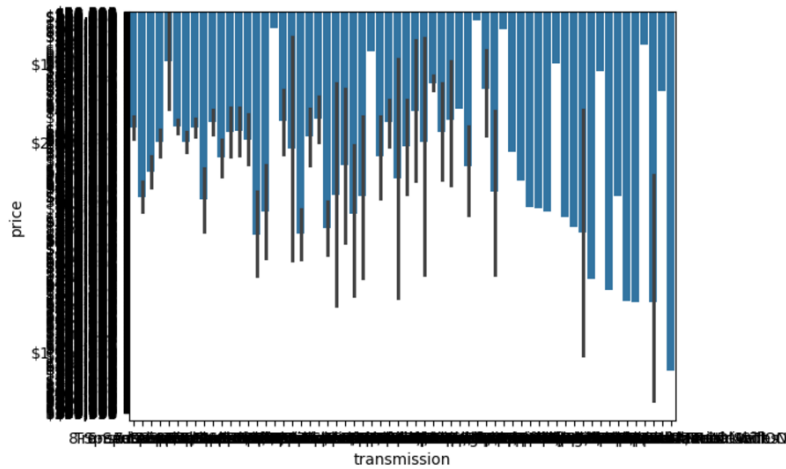


Figure 56: Transmission & Price Bar plot

This bar plot shows the relationship between price and transmission. However, it is very hard to visualize, and I did not include this in the main body. But Some transmissions are clearly more expensive than others. This is why I decided to keep this feature during the pre-processing.